Causal Rationale Extraction System

Technical report - NLP / Agentic division hackathon

1. Problem and approach

Given a corpus of agent-customer call transcripts with no outcome labels, the system answers natural language questions about why escalations, refunds and churn happen, backs each answer with statistics and cited dialogue turns, and supports contextual multi-turn follow-ups.

Questions such as 'why are escalations happening' are population-level analytical questions, not similarity lookups, so the system is not a retrieval chatbot. Offline stages label calls and extract behavioural features. The online stages select a subset of calls, compare it statistically with a control group, attribute the finding to specific turns by ablation, and only then give a language model the structured evidence to narrate. The language model is a writer constrained to the given evidence, not the source of the claim.

2. Data and quality findings

These figures were verified directly against the dataset.

Finding	Detail and handling
Corpus size	19,618 usable calls and 686,761 turns (mean 35 turns per call) after dropping 3 calls with fewer than 3 turns.
Constant domain field	The domain field has one value across all records and does not reflect call content, so it is excluded from filtering and feature engineering.
Non-standard speaker labels	1,332 calls (6.79 percent) use non-standard labels. Roles are resolved by first occurrence and an alternation check; unresolved calls are flagged, not guessed.
Leaked meta turns	A small number of calls contain a summary line that is not dialogue. These are stripped before feature extraction.

3. Architecture

Stage	Purpose	Key detail
0 Normalize	Fix data quality first	Strip meta turns, resolve speaker roles, flag uncertain calls.
1 Labeling	Silver outcome labels	Intent prior, zero-shot classification of reason_for_call, keyword scan. Confidence-tiered merge: zero-shot score 0.75 or above confirms alone; 0.5 to 0.75 needs one corroborating signal; otherwise low confidence.
2 Features	Turn and call signals	A cheap emotion model runs on all turns; the expensive zero-shot dialogue-act model runs only on negative turns. Aggregated to call-level features such as peak frustration, frustration slope, repetition and apology counts.
3 Indexing	Exemplar retrieval	reason_for_call plus first and last real turns embedded and stored in a vector database with all metadata.
4 Query parsing	Question to filter	Classify the outcome; match query terms against the 35 real intent values.
5 Comparison	What differs	Outcome calls versus the rest (intent-matched), tested per feature and ranked by effect size and significance.
6 Attribution	Which turns drive it	Leave-one-turn-out ablation on a lightweight classifier.
7 Synthesis	Readable explanation	Constrained prompt: statistics and cited turns only, 'associated with' rather than 'causes', under 200 words.

4. Statistical comparison (Stage 5)

- Treatment group: calls with the matching outcome. Control group: the remaining calls, intent-matched where possible.
- A t-test (numeric features) or chi-square test (categorical features) per feature, ranked by effect size and significance.
- If the treatment group has fewer than 30 calls after filtering, the system reports insufficient data rather than a result with false confidence.

5. Attribution and validation (Stage 6)

- A logistic regression is fitted on call-level behavioural features, outcome calls versus control. Call length and supervisor counts are excluded because they are proxies for the label, not behaviour.
- For 12 representative calls, spread across the predicted-probability range rather than the extremes, each turn is removed in turn, call features are recomputed, and the drop in predicted probability ranks the turns. The top three per call are kept as evidence.
- Validation: for each call, the drop from masking the top turn is compared with the expected drop from masking a random turn, using a one-sided Wilcoxon signed-rank test. Validation runs when at least 5 calls were ablated.

6. Conversational layer (Task 2)

Task 2 is a thin stateful session around the same pipeline. One structured model call classifies each message and extracts the change to the filter.

Message type	Behaviour
Refine	Merge the new constraint into the filter and re-run stages 4 to 7. Attribution is re-run only if the treatment group size changes by more than 20 percent; otherwise existing evidence is reused. Evidence is overwritten, never appended.
Drill down	No recomputation. Serve further cited turns from the evidence already computed.
New topic	Reset filter, factors and evidence. History is kept for the audit trail only.
History and small groups	History holds only query, one-line summary and timestamp per turn (capped). The 30-call floor also applies to narrow refinements.

7. Evaluation

Results not yet inserted. Run `python3 scripts/run_submission.py` and then rebuild this report so this section carries the measured numbers.

8. Limitations and guardrails

- Outcome labels are silver labels from heuristic and zero-shot signals, not ground truth. All findings are statistical associations and are worded as such.
- Attribution explains a lightweight proxy classifier, so cited turns show what drives the classifier's prediction, which is evidence for, not proof of, causation.
- Query parsing is keyword-based and intent matching is by substring, so unusual phrasings may not resolve to an outcome; the system then asks the user to name one.
- Every claim in a generated answer must cite a call and turn, and the model receives no material beyond the evidence bundle.

9. Reproducibility

Layer	Tools
Data	pandas, pyarrow, ijson (streaming JSON, parquet tables)
Models	facebook/bart-large-mnli (zero-shot), j-hartmann/emotion-english-distilroberta-base (emotion), all-MiniLM-L6-v2 (embeddings)
Storage and stats	ChromaDB, scipy, scikit-learn
Synthesis and serving	Anthropic API, FastAPI
Compute	Local CPU for logic and light models; Google Colab T4 for the two batched HuggingFace passes (labeling, turn features), each run once

Run `python3 src/explain.py` for the chat interface, `python3 scripts/run_submission.py` to regenerate the query results, and `python3 scripts/demo.py` for the scripted demo.